M2104 - Final review

1 Object concepts

Concept	Explanation	Example (Java)
Object	Individual in the code (variable, attribute, parameter) on which *operations* are defined (methods that can be called on the object) and containing *data* (attributes).	`String s = new String("Hello.");`
Operation	Treatment that can be performed by an object (instance method) or by a class (class method). The execution of an operation is made by a call (invocation) of the method, a message sent to the object (instance method) or the class (class method), with a list of *arguments*) (providing a value for each of the method's *parameters*).	`int l = s.length();`
Data	Value; class or instance *attribute* (field).	`private int x = 0;`
Encapsulation	A characteristic of the object paradigm: the fact that an object contains at the same time data and operations (on those data).	

Class	Definition of an object type; objects are instances of the class and have different values for the data that are instance attributes. Classes define the names and types of instance attributes, the signatures and codes of instance methods, and can also define class attributes and methods, not linked to any specific instance.	`public class A {`
		`  public static final int MAX = 100;`
		`  private final int x;`
		`  public A(int x) { this.x = x; }`
		`  public int getX() {return x; }`
		`  public static void methB() {}`
		`}`

Constructor	Code indicating how a new instance is constructed for a given class.	`public A(int x) { this.x = x; }`
Visibility	Accessibility or scope of items. Public: always visible, protected: visible in the package and outside for extending classes, package-private: visible only in the package, private: in the same class (including other instances of the class) only.	`public, protected, /* nil */, private`
Association	The fact for an object to contain a reference to one or several other objects as an attribute the type of which is a class or a container (array, collection, mapping…) of objects of that class.	`private A myA;`
Use	The fact for a (class or instance) method, or a constructor, in a class, to use another class (an object of that class and/or a method and/or a constructor). Associations and use links are the basis of the architecture of object programs.	`A.methB();`

Interface	Declaration of operations as method signatures constituting a contractual list of operations that must be performed by objects constituting instances of classes that conform to that contract.	`public interface I {`
		`  void m();`
		`}`

Abstraction	A characteristic of the object paradigm: the fact to rely upon the declaration of an operation (a method's signature, for example in an interface) instead of its definition (its code) in order to call it. Allows to (and forces to) ignore the specificities and foremost the choices in data structures and algorithms used in a given implementation, and thus to easily change that implementation.	`I thing = new A();`
		`thing.m(); // m declared in I`
		`  // defined in A`

Inheritance	A characteristic of the object paradigm: mechanism allowing to create a class extending another. The class inherits from all public, protected (and package-private, if defined in the same package) attributes and methods present in the extended class and can add its own, redefine (override) and overload methods.	`public class B extends A`

Sub-typing	The definition of a type Y more specific than an existing type X; all objects of type Y are also of type Y. The object paradigm allows sub-typing by the means of inheritance and interface implementation. An individual of a general type can thus be instanciated by a specific type.	`Set<X> myXs = new HashSet<>();`
Abstract method	Method declaration with no definition (code), that can be present in an abstract class. That method must be defined in the concrete (non-abstract) classes that extend it.	`public abstract void m();`

Re-definition	The act of providing a definition (some code) for a method already defined and accessible (visible) in a super-type (extended class), *overriding* it. The re-defined method always has the same signature as the method initially defined (and declared). This allows the class to have a different behaviour than the one it inherits from.	`public class C extends B {`
		`  @Override`
		`  public void m() {`
		`  System.out.println("C !");`
		`  }`
		`}`

Overload	The act of creating a method with the same name but a different parameter list as an existing method. This allows to have similar method calls.	`public interface I {`
		`  void m();`
		`  int m(int x);`
		`}`

Polymorphism	A characteristic of the object paradigm: mechanism that allows to have different behaviours for the same operation declared in a generic type. It can be performed by defining in different classes methods declared in a single interface, or declared abstract or defined in a single class.	`for (I is : myIs) {`
		`  is.m();`
		`}`

Package	Coherent set of types: classes and interfaces (and enumerations). Grouping types in a package allows them to share data and operations (using the package-private visibility in Java).	`package data;`

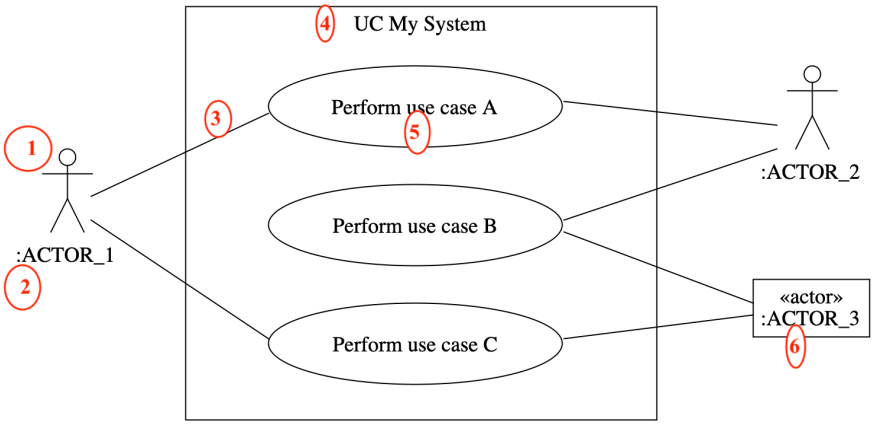
2 *Analysis*

The first analysis of a project is a modelling phase that allows to set the context of a system to be designed; it comes before design and is not (yet) concerned with the internal structure of the system.

Concept	Explanation
System in focus	The system in focus is: some software, an application, a server, a website, an information system, a database, &c. The analysis of needs only takes into account *one single entity* grouping all classes and objects and examining the *context of its use*: the possible interactions with the people and components that are *outside* only.
External actor	Person or hardware/software component that is *outside of the system in focus* and that *interacts with the system*. There can be human actors (users, administrators, operators) and non-human actors (web-services, APIs, captors) that send *external events* and/or receive *result events*.
External event	Event sent by an external event to the system in focus (action of the actor on the system, triggering an use case).
Result event	Event sent by the system in focus to an external actor (information of the system for an actor).
Use case	Feature performed by the system in focus given from the point of view of the user (external actor). Every use case: • is triggered by a single external event (trigger) sent by an external actor, • results in a series of treatments internal to the system in focus, • can, or not, end in one or several result events sent to one, or several, external actors.

3 *The UML use case diagram*

In the following UML use case diagram:

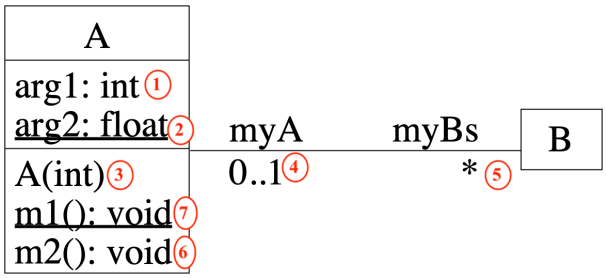


This is to what correspond each item number given in a circle (the number and the circle are not part of the diagram):

1. External, human actor: stick figure.
2. Name of the actor type: column and type in UPPERCASE (the name of the *person* would come before the column if it was pertinent, which is seldom the case in first-year diagrams).
3. Link (simple line with no annotation): when the actor sends an external event to the system or receives a result event from the system for the use case (nothing allows to distinguish the direction of the information flow in a use case diagram, that specification should be given elsewhere, in a review of all events for example).
4. Diagram title. In English: *UC System*.
5. Use case. A verb phrase (with an action verb that describe *what the system does for the user*, e. g., *Save the document as a file* is a use case). There are *precisely* as many use cases as there are external events.
6. External, non-human actor: rectangle box with the *actor* stereotype.

4 *The UML class diagram*

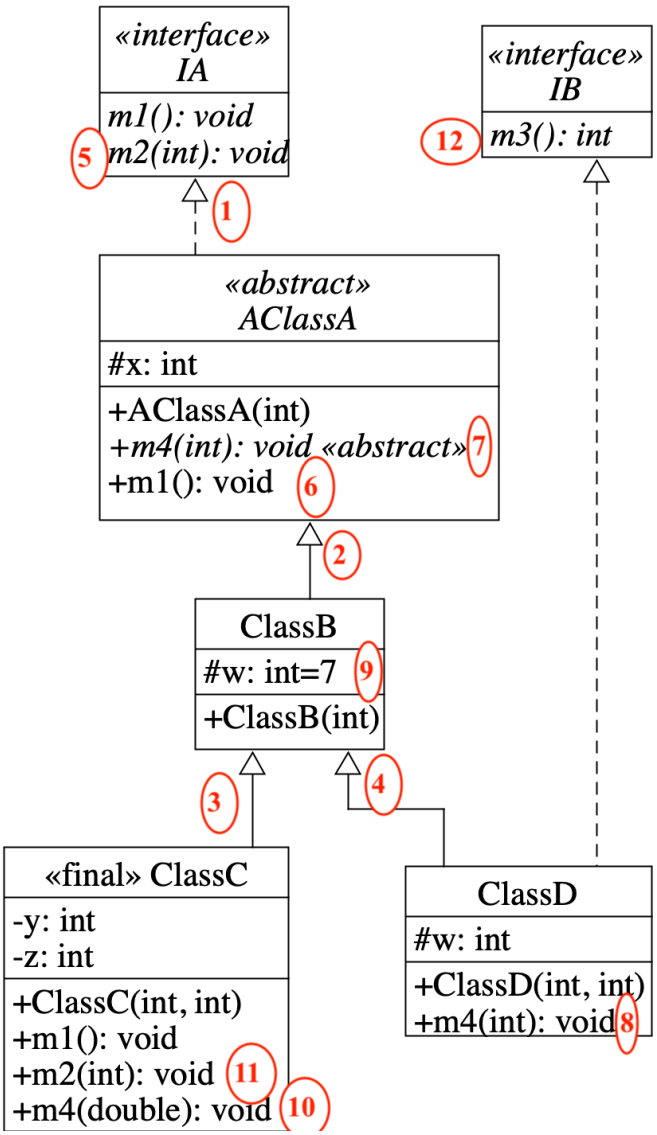
In the following UML class diagram (from the lesson):



The following facts can be deduced, for each of the items numbered in a circle (the number and circle are not part of the diagram):

1. Every object of type *A* has an instance attribute named *arg*1 of type *int*.
2. The *A* class has a class attribute (*static*) named *arg*2 of type *float*.
3. The *A* class has a constructor with one parameter of type *int*. It does not have any other constructor, implicit or not.
4. The *B* class is associated to the *A* class in the following fashion: every *B* refers to at most one *A*. The code of the *B* class thus contains an instance attribute of type *A* named *myA* (that can be null).
5. The *A* class is associated to the *B* class in the following fashion: evert *A* refer to some (zero, one, or more) *A*s. The code of the *A* class thus contains an instance attribute that is a *container* (array, collection…) of *B*s, named *myBs* (non null but possibly empty).
6. Every object of type *A* has an instance method *m*2() : *void*.
7. The *A* class has a class method *m*1() : *void*.

In the following UML class diagram (from the lesson):



The following facts can be deduced, for each of the items numbered in a circle (the number and circle are not part of the diagram):

1. The abstract class *AClassA* implements the *IA* interface and is thus a sub-type of it.
2. The *ClassB* class extends *AClassA* and is thus a sub-type of it, as well as a sub-type of *IA*.
3. The *ClassC* class extends *ClassB* and is thus a sub-type of it, as well as a sub-type of *AClassA* and of *IA*.
4. The *ClassD* class extends *ClassB* and is thus a sub-type of it, as well as a sub-type of *AClassA* and of *IA* (and also of *IB*).
5. The *ClassB* class (concrete class implementing *IA*) has a definition for a `m2(int): void` method.
6. The *AClassA* class (abstract class) has a definition for a `m1(): void` method.
7. The *ClassB* class (concrete class extending *AClassA*) has a definition for a `m4(int): void` method.
8. The *ClassD* class (extending *ClassB*) re-defines (overrides) the `m4(int): void` method defined in *ClassB* and declared in *AClassA*.
9. The *ClassC* and *ClassD* classes inherit the protected attribute *w* from *ClassB*.
10. The *ClassC* class overloads the *m*4 method inherited from *ClassB* and declared in *AClassA* with a different method, `m4(double): void`.
11. The *ClassC* class re-defines (overrides) the `m2(int): void` defined in *ClassB* and declared in *IA*.
12. The *ClassD* class defines a `m3(): int` method.